

SQL Part 8: Views, Indexes & Transactions Explained Simply — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

CREATE VIEW

```
CREATE VIEW open_loans AS SELECT l.loan_id, m.name, b.title, l.loan_date FROM loans AS l JOIN members AS m ON l.member_id = m.member_id JOIN books AS b ON l.book_id = b.id WHERE l.return_date IS NULL;
SELECT * FROM open_loans;
```

```
Expected result:
loan_id | name | title | loan_date
2 | Aisha | Wonder | 2026-02-10
5 | Carlos | Harry Potter | 2026-04-02
7 | Ben | Dog Man | 2026-05-01
8 | Dana | Holes | 2026-05-15
```

Using a view like a table

```
SELECT name, title FROM open_loans WHERE loan_date >= '2026-04-01' ORDER BY loan_date;
```

```
Expected result:
name | title
Carlos | Harry Potter
Ben | Dog Man
Dana | Holes
```

Views stay up to date

```
UPDATE loans SET return_date = '2026-05-20' WHERE loan_id = 7;
SELECT * FROM open_loans;
```

```
Expected result:
loan_id | name | title | loan_date
2 | Aisha | Wonder | 2026-02-10
5 | Carlos | Harry Potter | 2026-04-02
8 | Dana | Holes | 2026-05-15
```

Make a big table

```
CREATE TABLE visits AS WITH RECURSIVE n(x) AS (   SELECT 1 UNION ALL SELECT x + 1 FROM n WHERE x < 100000) SELECT x AS visit_id, (x % 5) + 1 AS member_id,        date('2026-01-01', '+' || (x % 365) || ' days') AS day FROM n;
SELECT COUNT(*) AS visits FROM visits;
```

Expected result:
visits
100000

EXPLAIN QUERY PLAN: SCAN

```
EXPLAIN QUERY PLAN SELECT * FROM visits WHERE day = '2026-03-15';
```

Expected result:
id | parent | notused | detail
2 | 0 | 0 | SCAN visits

CREATE INDEX

```
CREATE INDEX idx_visits_day ON visits(day);
EXPLAIN QUERY PLAN SELECT * FROM visits WHERE day = '2026-03-15';
```

Expected result:
id | parent | notused | detail
3 | 0 | 0 | SEARCH visits USING INDEX idx_visits_day (day=?)

Same answer, faster

```
SELECT COUNT(*) AS visits FROM visits WHERE day = '2026-03-15';
```

Expected result:
visits
274

BEGIN and COMMIT

```
BEGIN; UPDATE loans SET return_date = '2026-05-21'   WHERE loan_id = 5; INSERT INTO loans VALUES (9, 8, 3, '2026-05-21', NULL); COMMIT;
SELECT * FROM loans WHERE member_id = 3;
```

Error message (expected):
error
You can only execute one statement at a time.

Expected result:
loan_id | book_id | member_id | loan_date | return_date
4 | 4 | 3 | 2026-03-01 | 2026-03-15
5 | 1 | 3 | 2026-04-02 | NULL

ROLLBACK: the undo button

```
BEGIN; DELETE FROM loans; ROLLBACK;
SELECT COUNT(*) AS loans FROM loans;
```

Error message (expected):
error
You can only execute one statement at a time.

Expected result:
loans
8

CHECK and DEFAULT

```
CREATE TABLE ratings (    rating_id INTEGER PRIMARY KEY,    book_id INTEGER NOT NULL REFERENCES books(id),    stars INTEGER CHECK (stars BETWEEN 1 AND 5),    source TEXT DEFAULT 'desk' ); INSERT INTO ratings (rating_id, book_id, stars) VALUES (1, 7, 5), (2, 6, 4);
SELECT * FROM ratings;
```

Error message (expected):
error
You can only execute one statement at a time.

Error message (expected):

```
error
no such table: ratings
```

Step 4: Challenges (try first!)

C1. Create a view of members who have no email yet, then select from it.
C2. Add an index on loans(member_id) and check the query plan.
C3. Inside a transaction, delete all ratings, then undo it.

Answers

C1. Create a view of members who have no email yet, then select from it.

```
CREATE VIEW no_email AS SELECT name, city FROM members WHERE email IS NULL;
SELECT * FROM no_email;
```

```
Expected result:
name | city
Ben | Denver
Dana | Boston
```

C2. Add an index on loans(member_id) and check the query plan.

```
CREATE INDEX idx_loans_member ON loans(member_id);
EXPLAIN QUERY PLAN SELECT * FROM loans WHERE member_id = 2;
```

```
Expected result:
id | parent | notused | detail
3 | 0 | 0 | SEARCH loans USING INDEX idx_loans_member (member_id=?)
```

C3. Inside a transaction, delete all ratings, then undo it.

```
BEGIN;
DELETE FROM ratings;
ROLLBACK;
SELECT COUNT(*) FROM ratings;
```

```
Error message (expected):
error
no such table: ratings
```

```
Error message (expected):
error
no such table: ratings
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!